

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА
Институт искусственного интеллекта
Кафедра проблем управления

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине **Программное обеспечение мехатронных и робототехнических систем**

Тема лабораторной работы: «Отладка программного обеспечения робототехнических систем с использованием виртуального моделирования»

Позднева А. И. _____
Студенты группы: КРБО-01-23 Лазарева Д. Г. _____
Котова А. В. _____
Преподаватель: ст. преподаватель Морозов А. А. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

Цель работы: получение навыков моделирования объекта управления в промышленных системах автоматического управления и создание функциональных блоков.

Задание: создать виртуальную систему управления (рис. 1), включающую: модель объекта управления (рис. 2), ПИ-регулятор (рис. 3), сумматор и обратную связь. Передаточная функция объекта:

$$W = \frac{1}{k_e(T_m s+1)}$$

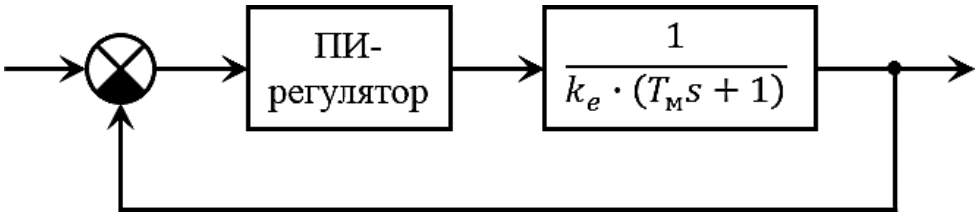
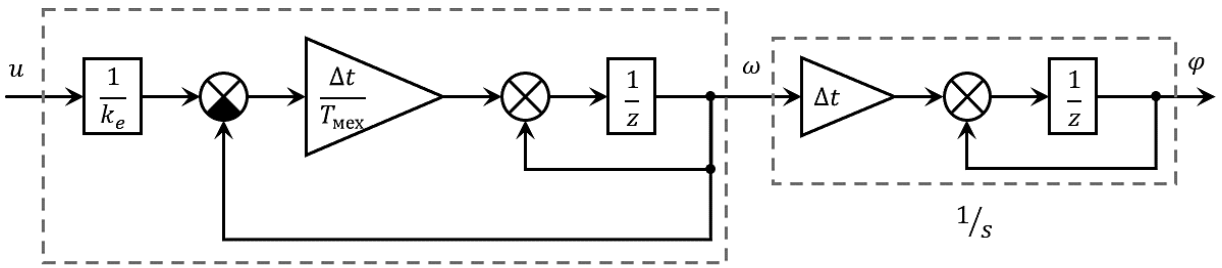


Рис. 1 - Структура системы управления



$$W = \frac{1}{k_e \cdot (T_м s + 1)}$$

Рис. 2 - Структура объекта управления

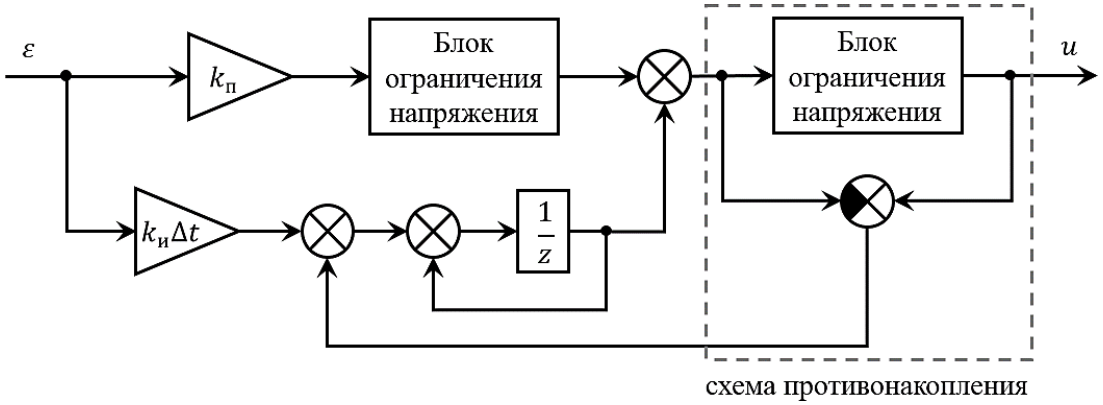


Рис. 3 - Структура ПИ-регулято

ХОД РАБОТЫ

Лабораторная работа была выполнена в программной среде Automation studio без конфигурации и оборудования, но с объектами ANSI C Program и ANSI C Library Motor Control.

Были созданы 3 функциональных блока - FB_Integrator, FB_Motor, FB_Regulator.

FB_Integrator - выполняет роль модели интегратора. На входе - значения интегрирующего звена, на выходе - соответственное значение с интегрирующего звена. Описание параметров представлено в Таблице 1.

Блок интегратора заменяет непрерывное интегрирование численным методом прямоугольников.

Таблица 1. Параметры функционального блока FB_Integrator

Конфигурация	Имя	Тип данных	Описание
вход	in	REAL	вход интегрирующего звена
выход	out	REAL	выход интегрирующего звена
внутреннее состояние	dt	REAL	шаг расчета [с]
внутреннее состояние	prev_value	VAR	Предыдущее состояние

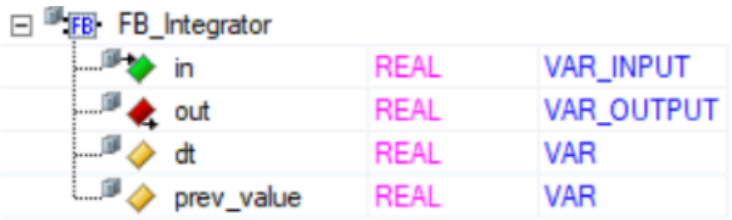


Рис. 4 - Переменные integrator в приложении

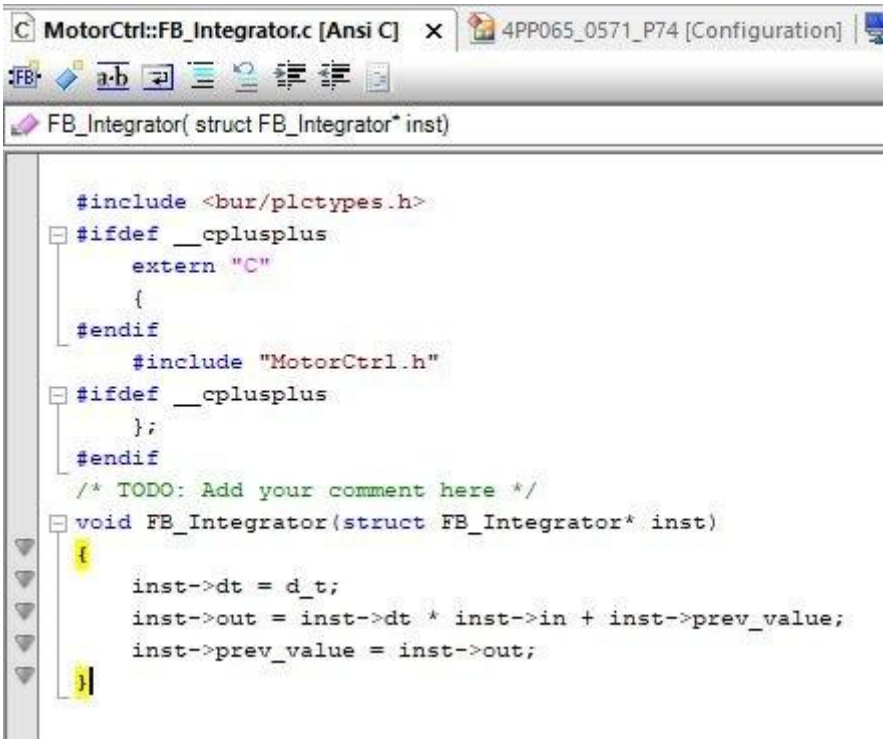


Рис. 5 - Блок интегратора

Следующий блок, FB_Motor, описывает математическую модель двигателя. Он как бы программирует физику ДПТ на основе дифференциального уравнения динамики вращения. Вход - напряжение, выход - частота вращения и положения. Его параметры представлены в Таблице 2. Расчет на выходе происходит в соответствии со схемой ДПТ на Рис 2.

Таблица 2. Параметры функционального блока FBMotor

Конфигурация	**Имя**	**Тип данных**	**Описа ние**
вход	u	REAL	входное напряжение [В]
выход	w	REAL	частота вращения [об/мин]
выход	phi	REAL	положение [рад]
внутреннее состояние	integrator	FB _Integrator	интегратор
внутреннее состояние	Tm	REAL	электромеханическая постоянная времени [с]
внутреннее состояние	ke	REAL	постоянная ЭДС двигателя [В•мин/об]
внутреннее состояние	dt	REAL	шаг расчета [с]

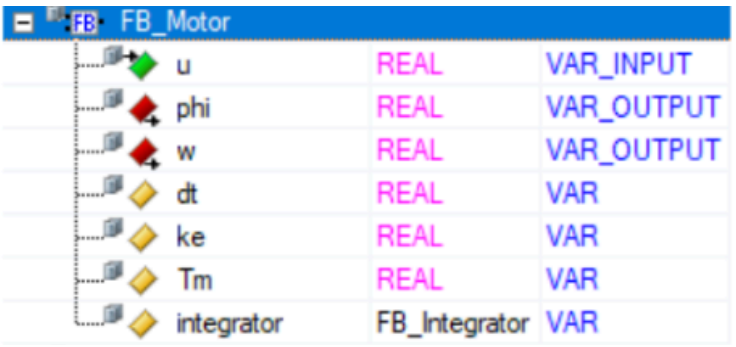


Рис. 6 - Переменные motor в приложении

```
#include <bur/plctypes.h>
#ifdef __cplusplus
    extern "C"
    {
#endif
    #include "MotorCtrl.h"
#ifdef __cplusplus
    };
#endif
/* TODO: Add your comment here */
void FB_Motor(struct FB_Motor* inst)
{
    REAL a = inst->u / inst->ke - inst->w;
    REAL b = a / inst->Tm;

    inst->integrator.in = b;
    FB_Integrator(&inst->integrator);

    inst->w = inst->integrator.out;
}
```

Рис. 7 - Блок двигателя постоянного тока

FB_Regulator - выполняет функции ПИ-регулятора с защитой от интегрального насыщения. На вход подаётся рассогласование между реальным воздействием и реальной скоростью вала ДПТ. На выходе - напряжение. Описание параметров представлено в Таблице 3.

Таблица 3. Параметры функционального блока FB_Regulator

Конфигурация	Имя	Тип данных	Описание
вход	e	REAL	рассогласование между задающим воздействием и реальной скоростью вращения вала ДПТ [об/мин]
выход	u	REAL	напряжение, подаваемое на вход ДПТ [В]
внутреннее состояние	k_p	REAL	пропорциональный коэффициент регулятора
внутреннее состояние	k_i	REAL	интегральный коэффициент регулятора
внутреннее состояние	integrator	FB_Integrator	интегратор
внутреннее состояние	iyOld	REAL	хранение предыдущего значения схемы противонакопления
внутреннее состояние	max_abs_value	REAL	граница блока ограничения [В]
внутреннее состояние	dt	REAL	шаг расчета [с]

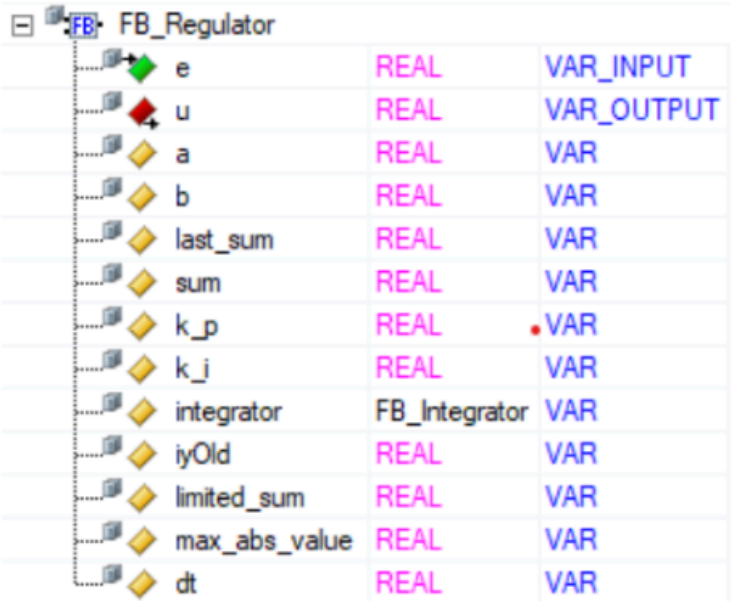


Рис.8 - Переменные regulator в приложении

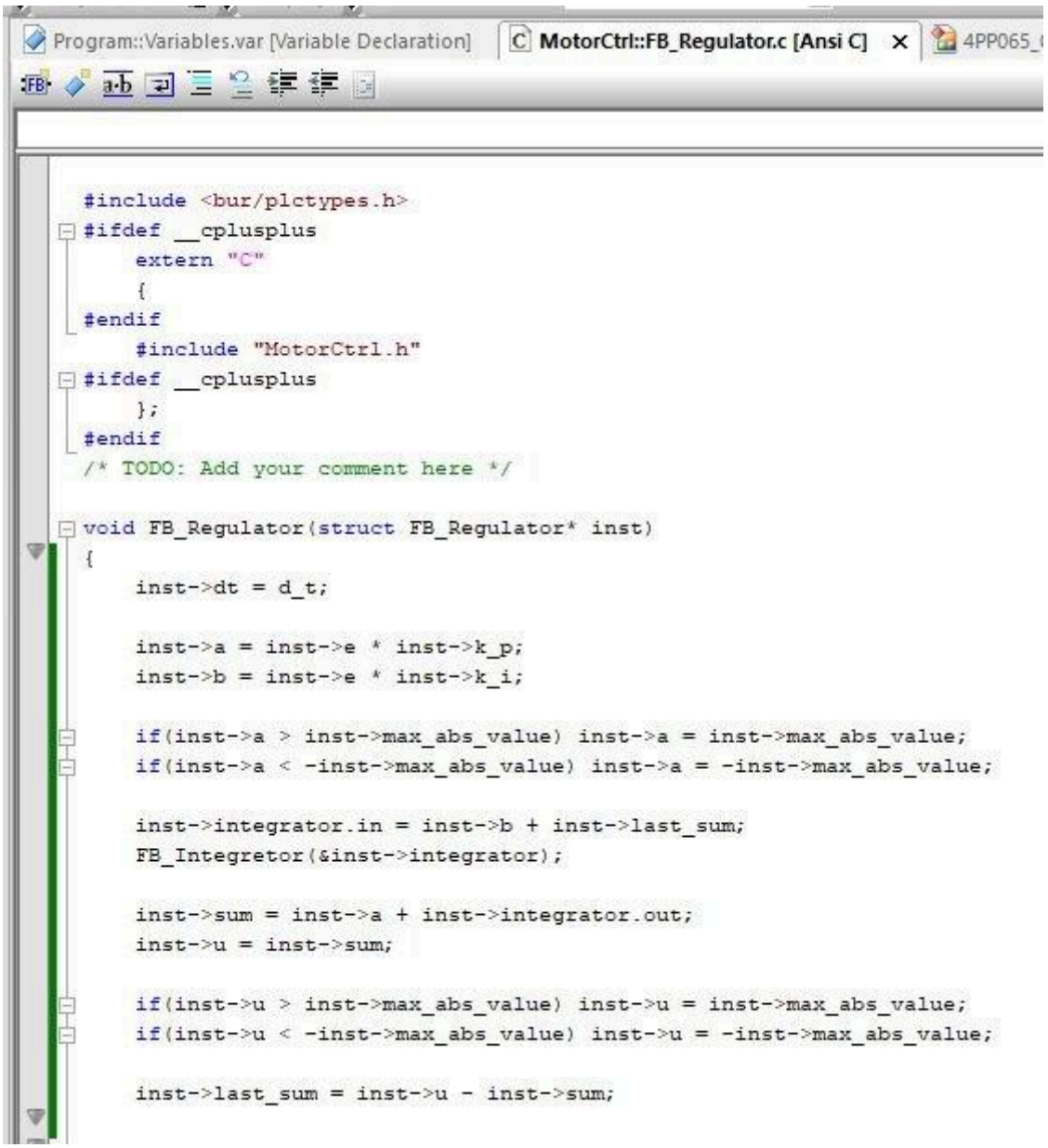


Рис.9 - Блок регулятора

Затем были созданы экземпляры объектов в глобальных переменных программы.

Name	Type	Reference	Constant	Retain	Replicable
enable	BOOL	☐	☐	☐	☑
regulator	FB_Regulator	☐	☐	☐	☑
e	REAL	☐	☐	☐	☑
counter	USINT	☐	☐	☐	☑
speed	REAL	☐	☐	☐	☑
fb_motor1	FB_Motor	☐	☐	☐	☑
fb_motor2	FB_Motor	☐	☐	☐	☑

Рис.10 - Переменные основной части программы

В файле Init.c были записаны все переменные. Возьмем их из файла motor: dt= 0.002; k_i = 0.16; k_p = 0.0064; pwm_period = 200; Tj = dt * 3=0.006; Tm = 0.0064 / 0.16 = 0.04; ke = k_i * Tj = 0.16*0.006 = 0.00096. ГЛЯНУТЬ

В итоге в системе управления в основном цикле Cyclic.c представлены 3 функциональных блока с программой, подающей на систему управления ступенчатое воздействие.

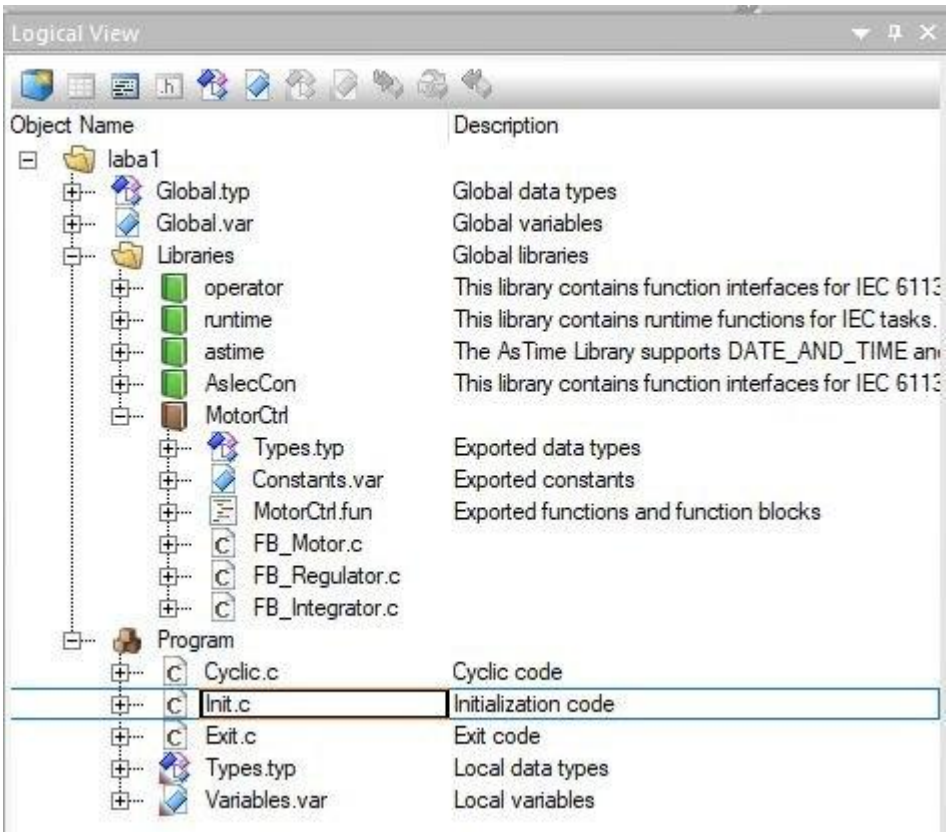


Рис. 11 - Параметры программы

С помощью инструмента Trace были получены графики входного воздействия, двигателя без регулятора и двигателя с регулятором.

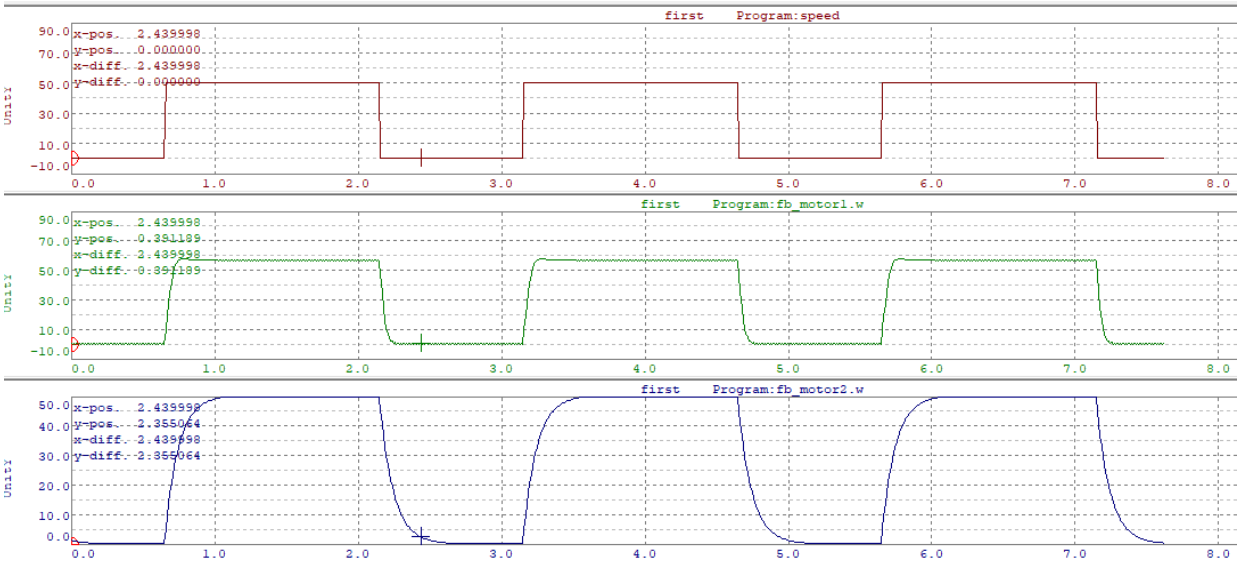


Рис. 12 - График уставки, скорости вращения двигателя без регулятора и скорости вращения двигателя с настроенным регулятором.

ВЫВОД

В ходе лабораторной работы были реализованы 3 функциональных блока, написана программа, совмещающая в себя созданные блоки. Были разработаны системы управления двигателем с регулятором и без. Регулятор позволяет контролировать некоторые параметры переходного процесса(время прихода в установившийся режим). Таким образом система с регулятором входит в установившийся режим за указанное нами время

Приложение А (cyclic)

```
#include <bur/plctypes.h>

#ifdef _DEFAULT_INCLUDES
    #include <AsDefault.h>
#endif



void _CYCLIC ProgramCyclic(void)
{
    if(enable){
        if(counter == 250){
            speed = 0;
            counter = 0;
```

```
        }
        if(counter == 100){
            speed = 50;
        }
        counter++;

        regulator.e = speed - fb_motor1.w;
        FB_Regulator(&regulator);

        fb_motor1.u = regulator.u;

        FB_Motor(&fb_motor1);

        fb_motor2.u = speed * fb_motor2.ke;
        FB_Motor(&fb_motor2);
    }
}
```

Приложение B (Init)

```
#include <bur/plctypes.h>

#ifdef _DEFAULT_INCLUDES
    #include <AsDefault.h>
#endif


void _INIT ProgramInit(void)
{
    fb_motor1.Tm = 0.1;
    fb_motor1.ke = 0.24;
    fb_motor1.u = 0.1;

    fb_motor2.Tm = 0.1;
    fb_motor2.ke = 0.24;
    fb_motor2.u = 0.1;

    speed = 0;
    counter = 0;
}
```

Приложение C (Motor)

```
#include <bur/plctypes.h>
```

```
#ifdef  cplusplus
extern "C"
{
#endif
#include "MotorCtrl.h"
#ifdef  cplusplus
};
#endif

/* TODO: Add your comment here */
void FB_Motor(struct FB_Motor* inst)
{
    REAL a = inst->u / inst->ke - inst->w;
    REAL b = a / inst->Tm;

    inst->integrator.in = b;
    FB_Integrator(&inst->integrator);

    inst->w = inst->integrator.out;
}
```

Приложение D (Regul)

```
#include <bur/plctypes.h>
#ifdef  cplusplus
extern "C"
{
#endif
#include "MotorCtrl.h"
#ifdef  cplusplus
};
#endif

/* TODO: Add your comment here */
void FB_Regulator(struct FB_Regulator* inst)
{
    inst->dt = d_t;
    inst->a = inst->e * inst->k_p;
    inst->b = inst->e * inst->k_i;

     inst->a = inst->a > inst->max_abs_value ? inst->max_abs_value :
inst->a;
     inst->a = inst->a < -inst->max_abs_value ? -inst->max_abs_value
```

```
: inst->a;

          inst->integrator.in = inst->b + inst->last_sum;
          FB_Integrator(&inst->integrator);

          inst->sum = inst->a + inst->integrator.out;
          inst->u = inst->sum;
           inst->u = inst->u > inst->max_abs_value ? inst->max_abs_value :
inst->u;
           inst->u = inst->u < -inst->max_abs_value ? -inst->max_abs_value
: inst->u;

          inst->last_sum = inst->u - inst->sum;
      }
```

Приложение E(Integrator)

```
      #include <bur/plctypes.h>
      #ifdef  cplusplus
      extern "C"
      {
      #endif
      #include "MotorCtrl.h"
      #ifdef  cplusplus
      };
      #endif

      /* TODO: Add your comment here */
      void FB_Integrator(struct FB_Integrator* inst)
      {
          inst->dt = d_t;
          inst->out = inst->dt * inst->in + inst->prev_value;
          inst->prev_value = inst->out;
      }
```